OS Tools – Installation Manual

Document Title: OpenSource Tools – Installation Manual
Purpose: VLSI Foundation Teaching Aid
Version: v0.1

1 Basic Pre-requisites: 3
1.1 System / OS 3
1.2 Hardware 3
1.3 Software / build environment 3
2 Ubuntu – specific pre-requisite packages 4
3 Xschem 4
3.1 Purpose 4
3.2 Source / reference 4
3.3 Pre- installation 4
3.4 Download & Installation Steps: 5
3.5 Post Installation Check 5
4 NGSPICE 5
4.1 Purpose 5
4.2 Source / references 5
4.3 Pre-installation 5
4.4 Installation 6
4.5 Post-installation 6
5 GAW Waveform Viewer Integration 7
5.1 Purpose 7
5.2 Source / references 7
5.3 Pre-installation 7
5.4 Installation 7
5.5 Post-installation 7
6 Installing Magic VLSI 8
6.1 Purpose 8
6.2 Source / references 8
6.3 Pre-installation 8
6.4 Installation 8
6.5 Post-Installation 8
7 Troubleshooting Guidelines and Common Issues 9

7.1 Read the Exact Error Message Carefully 9
7.2 Verify the Current Working Directory 9
7.3 Refresh Package Information Before Installing Dependencies 9
7.4 Install Failed Packages Separately 10
7.5 5. Repair Broken Package States 10
7.6 Check Whether the Tool Was Installed but Not Found in Terminal 10
7.7 Check Library Path Issues When the Command Exists but the Tool Still Fails 11
7.8 Use make and make install Correctly 11
7.9 Inspect config.log When ./configure Fails 11
7.10 Confirm That GUI Support Is Working 11
7.11 Reload the Shell After Editing .bashrc 12
7.12 Remove Older Conflicting Installations if Necessary 12
7.13 Do Not Skip the Prerequisite Installation Step 12
7.14 Check Internet and Repository Access During Clone or Download 12
7.15 Handle Permission Errors Carefully 13
7.16 Watch for Typing and Copy-Paste Errors 13
7.17 Verify Each Tool Immediately After Installation 13
7.18 Do Not Restart the Entire Flow Unnecessarily 14
7.19 Scroll Back and Identify the First Real Failure 14
7.20 Information Required Before Asking for Support 14
7.20.1 Reporting: 14
7.20.2 Minimum Self-Check Before Reporting an Error 14

1 Basic Pre-requisites:

1.1 System / OS

- **Ubuntu 20.04 LTS or newer**
- **64-bit system**
- **Internet connection** for package download, git clone, and PDK/tool dependencies
- **User with sudo privileges**

1.2 Hardware

- **RAM:** minimum **8 GB**, recommended **16 GB**
- **Free disk space:** minimum **30-40 GB**,
- **Processor:** dual-core minimum,

1.3 Software / build environment

- **Terminal familiarity** with basic Linux commands
- **Git installed**
- **Python 3 installed**
- **build-essential / compiler tools**
  (gcc, g++, make)
- **cmake**
- **autoconf / automake / libtool**
- **pkg-config**
- **bison and flex**
- **X11 / GUI support enabled**

2 Ubuntu – specific pre-requisite packages

```
sudo apt-get update
mkdir -p eda_tools
cd eda_tools
sudo apt-get install build-essential clang bison flex \
        libreadline-dev gawk tcl-dev libffi-dev git \
        graphviz xdot pkg-config python3 libboost-system-dev \
        libboost-python-dev libboost-filesystem-dev zlib1g-dev \
        make m4 tcsh csh libx11-dev gperf  tcl8.6-dev tk8.6 tk8.6-dev \
        libxmp4 libxpm-dev  libxcb1 libcairo2  \
    libxrender-dev libx11-xcb-dev libxaw7-dev freeglut3-dev automake yosys
sudo apt-get updat+e
sudo apt-get -y install libtool
```

3 Xschem

3.1 Purpose

xschem is an open-source schematic capture tool used for drawing circuits and generating netlists for simulation. It is commonly used along with ngspice for analog/custom design workflows. A schematic editor for VLSI/ASIC/analog custom designs with Spice, Verilog, and VHDL netlisting support.

3.2 Source / reference

Official project references:

- xschem project page / manual
- Source repository dcxsf+w

3.3 Pre- installation

Make sure the prerequisite packages from **Step 1** are installed. Also note that the official install page warns that if an older packaged version of xschem is already present in the system, it should be removed first to avoid conflicts with the source-built version. xschem is a GUI-based tool, so it requires a working desktop/X11 environment. If you are using VirtualBox or a remote Linux setup, ensure that GUI display support is available.

3.4 Download & Installation Steps:

```
cd ~/eda_tools/
git clone https://github.com/StefanSchippers/xschem.git xschem-src
cd xschem-src
./configure
make
sudo make install
cd ..
```

3.5 Post Installation Check

After installation, verify that xschem is accessible, If installed correctly, the xschem GUI should launch. also verify the executable path using 'which' command.
If you find any path issues, please refer troubleshooting section of this manual.

```
xschem
which xschem
```

4 NGSPICE

4.1 Purpose

ngspice is an open-source circuit simulator used for analog, mixed-signal, and device-level simulation. It is widely used in open-source IC design flows along with schematic tools such as xschem. The official ngspice documentation also covers compiling from source and using Git sources.

4.2 Source / references

- Official Home Page
- User Manual
- Installation Release

4.3 Pre-installation

Before starting the installation, ensure that the prerequisite packages from **Step 1** are already installed. In this flow, ngspice is built from source using the public Git repository. Your command set also requires **adms** before configuration, because --enable-adms is included in the configure options. The ngspice manual and documentation indicate that compiling from source is a standard supported flow on Linux.

```
sudo apt-get install adms
```

4.4 Installation

Run the following commands to clone, configure, build, and install ngspice. You may also note that if no --prefix is provided, installation typically goes under /usr/local/{bin,share,man,lib}; this is consistent with standard source builds and is also noted in ngspice's installation guidance.

```
cd ~/eda_tools/

git clone https://git.code.sf.net/p/ngspice/ngspice ngspice_git

cd ngspice_git

mkdir release

sudo apt-get install adms

./autogen.sh

cd release

../configure --with-x --enable-xspice --disable-debug --enable-cider --with-readline=yes --enable-openmp --enable-adms

sudo make

sudo make install

cd ..
```

4.5 Post-installation

After installation, verify that ngspice is available from the terminal:

```
which ngspice
ngspice
```

Troubleshooting note: If the terminal reports command not found even after successful installation, check whether the install location is included in the system PATH. A common case for manual source builds is that /usr/local/bin is not active in the current shell environment. In that case, add the path in ~/.bashrc and reload it. This is a practical Ubuntu troubleshooting note rather than an ngspice-specific requirement.

5 GAW Waveform Viewer Integration

5.1 Purpose

GAW (Gtk Analog Wave Viewer) is a Linux waveform viewer used for displaying analog waveforms from simulator output data. Its project page describes it as a tool for viewing sampled analog data, including simulator-generated waveform files.

5.2 Source / references

- Official Home Page

5.3 Pre-installation

Before installation, make sure GTK development support is available, since GAW is based on the Gtk toolkit. The project page explicitly states that GAW is based on Gtk, so installing the required GTK development package before building is appropriate.

```
sudo apt install libgtk-3-dev
```

5.4 Installation

Download link:

After downloading the GAW source archive, extract and install it using:

```
tar zxvf gaw-yyyymmdd.tar.gz
cd gaw-yyyymmdd
./configure
sudo make
sudo make install
```

5.5 Post-installation

After installation, verify that GAW is accessible:

```
which gaw
gaw
```

Troubleshooting note: GAW is optional in the flow, but it is useful when you want a standalone viewer for analog simulation waveforms. If gaw does not launch after installation, first check the executable path and then verify whether the install location has been added to the system PATH.

6 Installing Magic VLSI

6.1 Purpose

Magic is an open-source VLSI layout editor used for custom IC layout creation, design-rule checking, extraction, and layout visualization. In the open-source IC flow, it is commonly used along with xschem, ngspice, and SKY130/Open PDK-based design work.

6.2 Source / references

Open Circuit Design references for Magic

installation is being done directly from source to obtain a recent working version for the Linux-based open-source flow.

6.3 Pre-installation

Before installing Magic, ensure that all prerequisite packages from **Step 1** are already installed.

GUI support must be available, since Magic is a graphical tool.

A working build environment is required (gcc, make, tcl/tk, X11-related libraries, etc.), which are already covered in the prerequisite installation step.

6.4 Installation

Run the following commands:

```
cd ~/eda_tools/
git clone git://opencircuitdesign.com/magic
cd magic
./configure
make
sudo make install
cd ..
```

6.5 Post-Installation

After installation, verify that Magic is correctly installed:

which magic
magic

If installed correctly, the Magic GUI should open.

7 Troubleshooting Guidelines and Common Issues

This section summarizes the most common issues encountered during installation and setup of the open-source EDA tools used in this guide, including xschem, ngspice, Magic, and KLayout. Students are strongly advised to review this section carefully before reporting an installation problem. In most cases, failures are caused by missing dependencies, incorrect working directory, environment path issues, or skipped setup steps.

7.1 Read the Exact Error Message Carefully

Do not conclude that a tool “is not working” without checking the exact terminal output. The most useful information is usually contained in the final meaningful error line.

Typical examples include:

- E: Unable to locate package ...
- No package '...' found
- configure: error: ...
- make: *** Error ...
- command not found
- Permission denied
- No such file or directory

Always identify the precise error before attempting a fix.

7.2 Verify the Current Working Directory

Many installation errors occur because commands are executed from the wrong directory. Before running ./configure, make, or make install, confirm the current location.

Use:

```
pwd
ls
```

This is especially important after using multiple cd commands. If unsure, verify the folder contents before proceeding.

7.3 Refresh Package Information Before Installing Dependencies

If Ubuntu reports that a package cannot be found, first refresh the package index and then retry the installation.

```
sudo apt-get update
```

This should be done before installing prerequisite packages and again whenever package lookup issues occur.

7.4 Install Failed Packages Separately

If a long apt-get install command fails, do not assume the full setup is broken. Instead, identify the failed package and install it separately.

Example:

sudo apt-get install adms

If the package name appears incorrect or unavailable, search for it using:

apt search <package-name>

This helps isolate the real dependency issue quickly.

7.5 5. Repair Broken Package States

If package installation was interrupted or Ubuntu reports dependency conflicts, run the following recovery commands:

sudo apt --fix-broken install

sudo dpkg --configure -a

After that, retry only the failed installation step.

7.6 Check Whether the Tool Was Installed but Not Found in Terminal

A very common issue in source-based installation is that the tool is installed successfully, but the terminal still reports command not found. This usually indicates that the installation path is not included in the system environment PATH.

Check whether the command is visible:

which xschem

which magic

which ngspice

which klayout

If no path is returned, verify the current PATH:

echo $PATH

If required, add the install location to ~/.bashrc.

Example:

gedit ~/.bashrc

Then add a line such as:

export PATH=$PATH:/usr/local/bin

If the tool was installed in a custom user directory, add that directory instead. Then reload the shell environment:

source ~/.bashrc

This is one of the most common causes of post-installation confusion.

7.7 Check Library Path Issues When the Command Exists but the Tool Still Fails

In some cases, the executable is found, but the required shared libraries are not. This can happen in manual source builds.

A common fix is to update the library path:

export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib

source ~/.bashrc

This is less common than a PATH issue, but should be checked when a tool launches with library-related errors.

7.8 Use make and make install Correctly

Recommended practice is:

- run ./configure as a normal user
- run make as a normal user
- use sudo only for make install

Example:

./configure

make

sudo make install

Using sudo unnecessarily during build steps can create permission or ownership problems.

7.9 Inspect config.log When ./configure Fails

If a ./configure step fails, do not guess the cause. Check the generated config.log file in the source directory:

cat config.log

This file often reveals the actual missing header, library, compiler issue, or unsupported configuration option.

7.10 Confirm That GUI Support Is Working

Tools such as **xschem**, **Magic**, and **KLayout** are graphical applications. If the command runs but no window appears, the issue may be related to the Ubuntu graphical environment rather than the tool itself.

Check:

- whether Ubuntu Desktop is installed

- whether VirtualBox display settings are configured properly
- whether other GUI applications launch correctly

You may test with:

gedit

If basic GUI applications do not open, resolve the display issue first.

7.11Reload the Shell After Editing .bashrc

After adding environment variables or updating PATH entries in ~/.bashrc, the changes will not take effect automatically in the current terminal.

Run:

source ~/.bashrc

Alternatively, close the terminal and open a new session before testing the tool again.

7.12 Remove Older Conflicting Installations if Necessary

If a tool was previously installed through Ubuntu packages and is later installed again from source, version conflicts may occur. This can lead to:

- launching the wrong version
- missing expected features
- confusion about which binary is being used

Verify using:

which xschem

which magic

which ngspice

If needed, remove the older installation and repeat the source installation cleanly.

7.13 Do Not Skip the Prerequisite Installation Step

Most configure and build failures occur because the prerequisite packages were skipped or only partially installed. Always complete the prerequisite section fully before beginning installation of any tool.

7.14 Check Internet and Repository Access During Clone or Download

If git clone fails, verify:

- internet connectivity
- correctness of the repository URL
- whether Git is installed properly

Useful checks:

```
git --version
```

```
ping google.com
```

Transient network issues can also interrupt downloads, so retry if required.

7.15 Handle Permission Errors Carefully

If a command fails with Permission denied, check whether:

- the file needs execute permission
- the step actually requires sudo
- earlier commands created root-owned files in the working directory

Where applicable, execution permission may be added using:

```
chmod +x ./configure
```

7.16 Watch for Typing and Copy-Paste Errors

A large number of setup failures are caused by simple mistakes such as:

- mistyped package names
- incorrect folder names
- missing characters
- extra spaces
- wrong case in file names
- using the wrong directory level after cd

Students should compare commands carefully with the manual before executing them.

7.17 Verify Each Tool Immediately After Installation

Do not wait until the end of the full setup to test everything. After each installation step, verify the tool immediately.

Example:

```
which xschem
xschem

which magic
magic

which ngspice
ngspice
```

This makes it easier to identify which specific step failed.

7.18 Do Not Restart the Entire Flow Unnecessarily

If one step fails, it is usually sufficient to fix that specific issue and rerun only the failed command or stage.

For example:

- if ./configure fails, fix dependencies and rerun ./configure
- if make fails, fix the build issue and rerun make
- if make install fails, rerun only make install

Avoid deleting and reinstalling everything unless truly required.

7.19 Scroll Back and Identify the First Real Failure

Terminal output can be long, and the final line is not always the root cause. If a build fails, scroll upward and locate the first meaningful error message. That is often the actual source of the problem.

7.20 Information Required Before Asking for Support

7.20.1 Reporting:

When reporting an installation issue, students should provide the following:

1. the exact command executed
2. the exact error message
3. output of pwd
4. output of which <toolname>
5. confirmation of whether source ~/.bashrc was run
6. whether the setup is on native Ubuntu or VirtualBox
7. a terminal screenshot, if possible

Without these details, troubleshooting becomes slow and ambiguous.

7.20.2 Minimum Self-Check Before Reporting an Error

Before escalating an installation problem, verify all of the following:

- the prerequisite installation step was completed fully
- the command was executed in the correct directory
- sudo apt-get update was run
- the exact error line was read carefully
- config.log was checked for configure-stage failures

- the PATH was verified using echo $PATH
- source ~/.bashrc was run after editing environment variables
- which <toolname> was checked
- GUI functionality was confirmed
- only the failed step was retried after fixing the issue

Important: Students must complete the checks in this section before requesting support for installation-related issues.